

Compact Restoring Divider

Area reduction for the RV32M hardware divider

The `picorv32_pcp_i_div` module in `picorv32.v` is the optional hardware divider enabled by `ENABLE_DIV`. It was rewritten into a more compact restoring divider. It computes the same RV32M results, that is `div`, `divu`, `rem` and `remu`, including divide-by-zero and the `INT_MIN / -1` overflow case, in the same roughly 33 cycles, but uses noticeably less FPGA area. On a nearly full iCE40 UP5K that area is what let the team fit the larger look-ahead cache and then the two-way associative version.

What the stock divider does

The divider that ships with PicoRV32 is correct but large. It holds the divisor pre-shifted left by 31 bits in a 63-bit register and shifts it right one place per iteration, while walking a 32-bit one-hot mask to pick off quotient bits. Each cycle it compares and subtracts across the full 63-bit divisor. The dividend and quotient sit in two further 32-bit registers.

```
reg [31:0] dividend; // 32 FF
reg [62:0] divisor; // 63 FF (pre-shifted, shifts right each cycle)
reg [31:0] quotient; // 32 FF
reg [31:0] quotient_msk; // 32 FF (one-hot iteration mask)
reg running, outsign; // 2 FF
                        // about 159 FF, 63-bit subtract/compare datapath
```

How the compact version works

The divisor stays a fixed 32-bit value and the partial remainder is shifted left one bit per step, pulling in the next dividend bit at the top. The dividend and quotient share a single 32-bit register `dq`, the dividend bits shifting out the top as quotient bits shift in the bottom. A six-bit step counter replaces the 32-bit one-hot mask.

```
reg [31:0] divisor_q; // 32 FF (held fixed, no shifting)
reg [32:0] rem_q; // 33 FF (running remainder, +1 bit for the shift)
reg [31:0] dq; // 32 FF (dividend out the top, quotient in the bottom)
reg [5:0] step; // 6 FF (binary step counter, not a one-hot mask)
reg running, outsign; // 2 FF
                        // about 104 FF, 33-bit subtract datapath
```

The inner loop is the whole design in a few lines:

```
// one restoring step
wire [32:0] rem_shifted = {rem_q[31:0], dq[31]}; // shift remainder left
wire [32:0] sub = rem_shifted - {1'b0, divisor_q}; // 33-bit subtract, not 63
wire ge = ~sub[32]; // no borrow means it fits
...
rem_q <= ge ? sub : rem_shifted; // keep the subtraction, or restore
dq <= {dq[30:0], ge}; // dividend out the top, quotient in the bottom
step <= step + 1'b1; // 6-bit counter, not a 32-bit one-hot mask
```

Signed operations are handled around that loop. The operands are made unsigned on the way in and the result sign is restored on the way out, so the inner loop is plain unsigned division. Divide-by-zero and the `INT_MIN / -1` overflow case need no special handling, as they fall out of the unsigned loop and the sign term. The `|rs2` guard stops a zero divisor from flipping the quotient sign.

```
// start: convert to unsigned (div/rem are the signed ops), record result sign
dq <= (instr_div||instr_rem) && pcp_i_rs1[31] ? ~pcp_i_rs1 : pcp_i_rs1;
divisor_q <= (instr_div||instr_rem) && pcp_i_rs2[31] ? ~pcp_i_rs2 : pcp_i_rs2;
outsign <= (instr_div && (pcp_i_rs1[31] != pcp_i_rs2[31]) && |pcp_i_rs2)
```

```

|| (instr_rem && pcpi_rs1[31]);
// after 32 steps: pick quotient or remainder and restore the sign
pcpi_rd <= (instr_div||instr_divu) ? (outsign ? -dq : dq)
          : (outsign ? -rem_q[31:0] : rem_q[31:0]);

```

The two changes that save the area are the shared `dq` register and the 33-bit subtraction instead of 63-bit, because the divisor is never pre-shifted.

How to build and measure it

The divider is part of the core, so it is exercised by every firmware run and by the divider unit testbench (see the Unit Testbenches document, run with `make div_test` from the `picorv32/` directory). To reproduce the area numbers below, build the full picosoc both ways with the same yosys and nextpnr seed and target frequency, with no cache, barrel shifter on, divider on, fast multiplier on and compressed ISA off, then read the `ICESTORM_LC` count from the synthesis report.

Results

Structure

	Stock divider	Compact divider
Divisor register	63 bits (pre-shifted)	32 bits (fixed)
Dividend + quotient	2×32 -bit regs	1 shared 32-bit reg
Iteration counter	32-bit one-hot mask	6-bit binary counter
Subtract / compare width	63-bit	33-bit
Total flip-flops	about 159 FF	about 104 FF
Latency	about 33 cycles	about 33 cycles
Result correctness	RV32M	identical

Measured iCE40 (UP5K) area

Build	ICESTORM_LC used	Utilisation (of 5280)
Stock divider	4754	90%
Compact divider	4589	86%

That is about 165 logic cells saved, roughly 3.1% of the device. The saving is larger than the 55-flip-flop figure suggests because on iCE40 each `ICESTORM_LC` bundles one LUT4 and one flip-flop, so the cell delta also picks up the combinational logic the flip-flop count never captures. Roughly half the win is registers with their feeder LUTs and the other half is combinational logic, mainly collapsing the 63-bit comparator and subtractor to 33 bits and removing the 32-bit one-hot shift register. Going from 90% to 86% utilisation on a nearly full UP5K is the headroom that helped place-and-route close timing and left room for the cache.

A couple of caveats. Synthesis numbers swing with the yosys and nextpnr version, the seed and the `abc` options, so a swing of 10 to 20 cells is noise. The rewrite also removed the `RISCV_FORMAL_ALTOPS` shortcut path that the original divider carried for riscv-formal runs. Functional RV32M behaviour is unchanged, and the divider unit testbench passes all directed and randomised checks.